Язык программирования Python

Условные операторы

Доцент кафедры ИВТ, к.т.н.
Проскурин Александр Викторович

Операции сравнения (1)

При реализации программ часто возникает необходимость в сравнении значений переменных с литералами или значениями других переменных.

Для этого в Python предусмотрено 6 операций сравнения. Результатом их выполнения является значение типа bool (**True** – истина (да) или **False** – ложь (нет)).

Например, для ответа на вопрос «Вы совершеннолетний?» необходимо сравнить свой возраст в годах с числом 18. При возрасте большем или равном 18 ответ будет «да» (True).

Операция	Описание	Примеры

==	Проверяет равны ли операнды между собой. Если они равны, то выражение становится истинным.	`>>> 5 == 5`
		`True`
		`>>> 1.0 == 1`
		`True`
		`>>> 6 == 44`
		`False`

!=	Проверяет равны ли операнды между собой. Если они НЕ равны, то выражение становится истинным.	`>>> var = 12`
		`>>> var != 12`
		`False`
		`>>> 1231 != 0.4`
		`True`

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Операции сравнения (2)

Операция	Описание	Примеры

>	Проверяет больше ли левый операнд чем правый. Если больше, то выражение становится истинным.	`>>> 53 > 23`
		`True`
		`>>> 1 > 1`
		`False`
		`>>> 432.5 > 500.1`
		`False`

<	Проверяет меньше ли левый операнд чем правый. Если меньше, то выражение становится истинным.	`>>> 5 < 51`
		`True`
		`>>> 6 < 4`
		`False`

>=	Проверяет больше или равен левый операнд, чем правый. Если больше, то выражение становится истинным.	`>>> 5 >= 5`
		`True`
		`>>> 6 >= 44`
		`False`

<=	Проверяет меньше или равен левый операнд, чем правый. Если меньше, то выражение становится истинным.	`>>> 32 <= 232`
		`True`
		`>>> 65 <= 9`
		`False`

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Использование операций сравнения

```
1  >>> a = 2
2  >>> b = 6
3  >>> a == b
4  False
5  >>> a < b
6  True
7  >>> # Сравнение также работает для строк и других типов:
8  ... 'Строка' == 'Строка'
9  True
10 >>> 'Кот' != 'Кошка'
11 True
12 >>> # Операции сравнения можно объединить в цепочки для проверки
13 ... # на вхождение значения в диапазон:
14 ... 0 < a < 10
15 True
16 >>> # Сравнивать можно только совместимые типы:
17 ... 'Three' < 4
18 Traceback (most recent call last):
19     File "<stdin>", line 1, in <module>
20 TypeError:'<' not supported between instances of 'str' and 'int'
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Логические операции (1)

Для объединения двух и более сравнений используются логические операции **И** (and) и **ИЛИ** (or). Также может применяться логическая операция **НЕ** (отрицание) (not) для инвертирования булевого значения.

Операция	Описание	Примеры

`and`	Логическая операция «И». Условие будет истинным если оба операнда истина	`True and True == True`
		`True and False == False`
		`False and True == False`
		`False and False == False`

`or`	Логическая операция «ИЛИ». Если хотя бы один из операндов истинный, то и все выражение будет истинным	`True or True == True`
		`True or False == True`
		`False or True == True`
		`False or False == False`

`not`	Логическая операция «НЕ». Изменяет логическое значение операнда на противоположное	`not True == False` `not False == True`

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Логические операции (2)

Следует отметить, что Python может представлять любой объект как булевое значение:

- Число 0 означает **False**, а любое другое – **True**.
- Пустые коллекции (строки, кортежи, списки, множества и т.д.) означают **False**. Если в коллекции есть хотя бы один элемент, то **True**.
- Специальный пустой объект None, который показывает, что значение «неизвестно» или «не существует», означает **False**.

Операции and и or вычисляются по короткой схеме и возвращают значение операнда, который определяет результат.

Например, если левый операнд логической операции ИЛИ истинен, то Python вернёт его значение и не будет проверять правый операнд (в этом нет смысла, т.к. ответ всё равно не изменится).

Аналогично для операции И – если левый операнд ложен, то его значение возвращается, а правый операнд не проверяется (истины теперь всё равно не добиться).

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Использование логических операций

```
1  >>> k = 2
2  >>> m = 6
3  >>> n = -5
4  >>> # Проверяем, что в k значение больше 0, а в m меньше 10:
5  ... k > 0 and m < 10
6  True
7  >>> # Проверяем, что хотя бы одна переменная положительная:
8  ... k > 0 or m > 0 or n > 0
9  True
10 >>> st = ''
11 >>> # Проверяем, пустая ли строка хранится в st. Нам нужно
12 ... # получить True, если она пуста:
13 ... not st
14 True
15 >>> # Демонстрация того, что and возвращает значение операнда:
16 ... m and k
17 2
18 >>> 0 and t
19 0
```

Сначала оба операнда представляются в виде булевых значений. Затем для and возвращается значение первого операнда, представленного как False, или второго, если оба операнда представлены как True.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Приоритет операций

Если в выражении используются несколько операций, то необходимо знать в каком порядке будет выполняться выражение.

Например, операция логического **И** имеет более высокий приоритет, чем операция логического **ИЛИ**. В таблице операции будут расположены от наиболее низкого приоритета, к более высокому. Операции, расположенные в одной ячейке, имеют одинаковый уровень приоритета.

`=`	`or`	`and`	`not`	`==, !=, <, <=, >=, >`	`+, -`	`*, /, //, %`	`**`

Низкий приоритет → Высокий приоритет

Стоит отметить, что для записи сложных выражений лучше использовать скобки. Такой подход поможет указать приоритет выполнения операций в явном виде.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Инструкции управления потока выполнения

Все инструкции, записанные в коде программы, выполняются последовательно, строка за строкой. Однако возможно изменение порядка выполнения инструкций, если будут встречены вызовы функций или методов, а также такие структуры управления, как **условные операторы (if)** или **операторы циклов (while, for)**.

В последних двух случаях в зависимости от истинности какого-либо условия будут выполняться так называемые блоки кода.

Блок кода (или вложенные инструкции) – это последовательность из одной или более инструкций, которые воспринимаются как одно логическое целое, располагаются с одинаковым отступом (обычно 4 пробела) от левого края и следуют после двоеточия.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Условный оператор if (1)

Вычислительный процесс называется **разветвляющимся**, если в зависимости от выполнения или не выполнения некоторого условия совершается либо одна, либо другая последовательность действий (ветвь вычислений).

Условный оператор if («если») является основным оператором проверки выполнения условия.

Для того, чтобы выполнить простую вложенную инструкцию, необходимо проверить условие на соответствие, использовав оператор **if** и прописав после него соответствующее условие:

```
var = 5 # объявляем переменную
# выполняем проверку условия
if var < 10:
    # если условие истинно, то выполняется
    # вложенная инструкция
    print('Переменная var меньше 10')
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

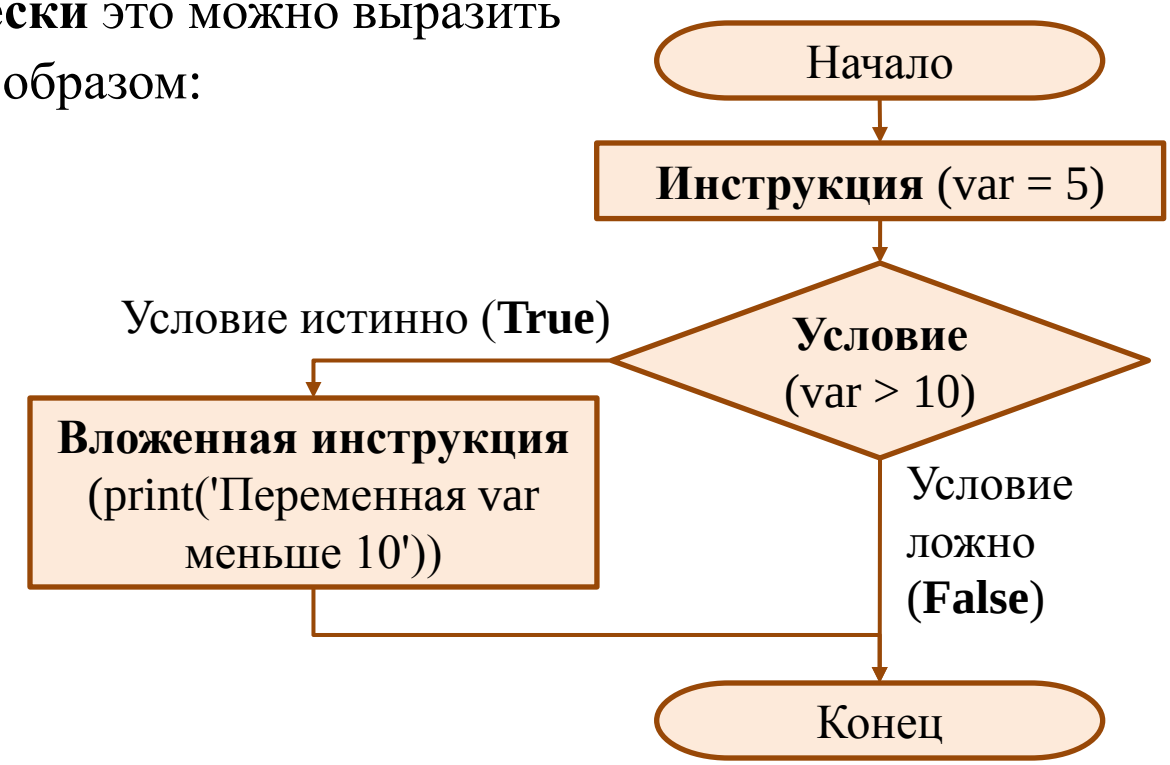
Условный оператор if (2)

Оператор **if производит проверку истинности выражения**, т.е. является ли результат выражения логической истиной (True) или же ложью (False).

Далее **выполняется вложенная инструкция, если результат выражения является True**.

Если результат выражения является False, тогда вложенная инструкция игнорируется.

Графически это можно выразить следующим образом:



© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Условный оператор else (1)

Оператор if позволяет выполнить одну или несколько вложенных инструкций, если результат выражения является истинной (True).

Часто возникает необходимость чтобы программа выполнила отличный набор инструкций в случае ложности условия (False). Для этого используется оператор **else** («иначе»).

Условный оператор else является продолжением оператора if и не используется отдельно:

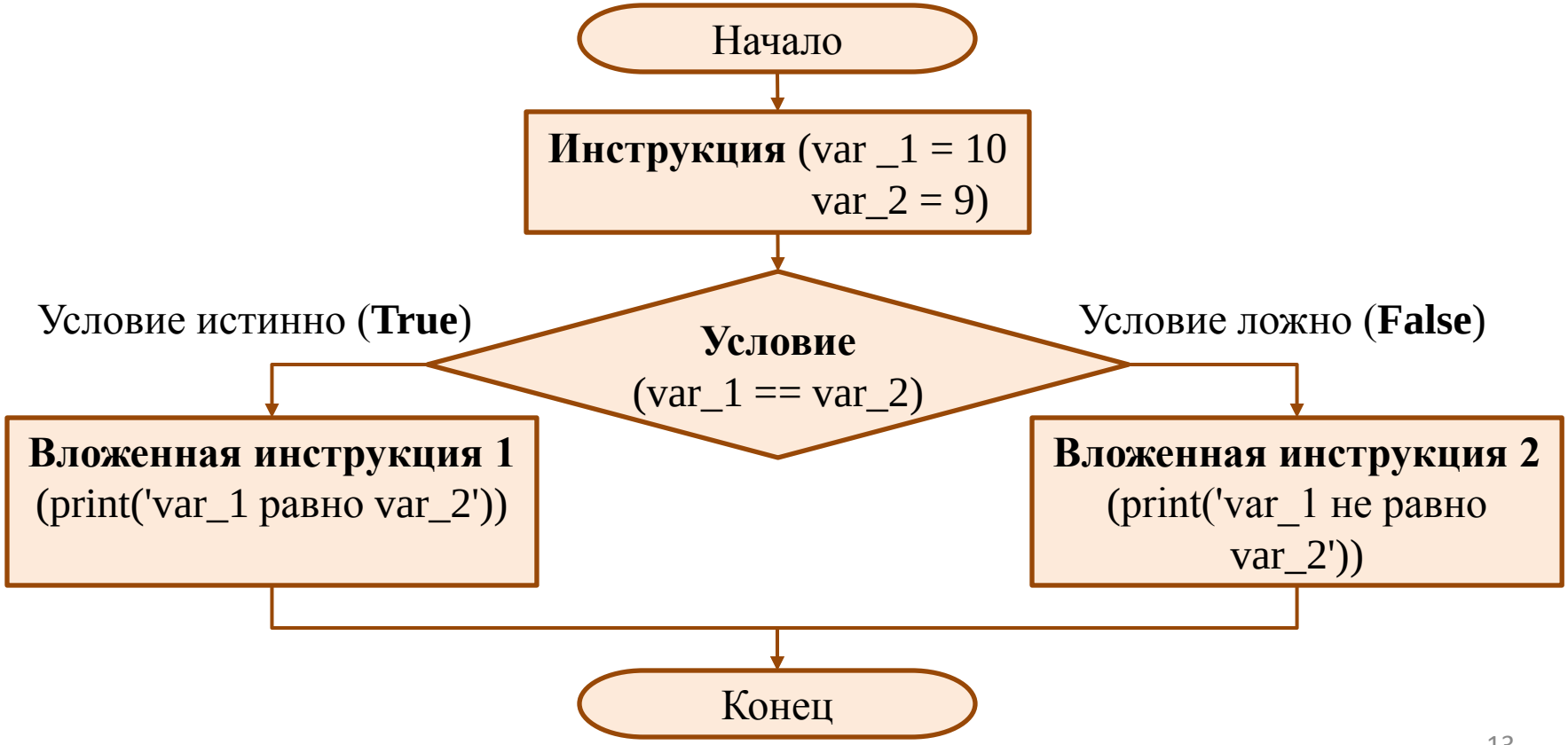
```
# объявляем переменные
var_1 = 10
var_2 = 9
# выполняем проверку условия
if var_1 == var_2: # если True
    print('var_1 равно var_2')
else: # иначе
# выполняется другая вложенная инструкция
    print('var_1 не равно var_2')
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Условный оператор else (2)

Оператор if производит проверку истинности условия.
Если условие истинно (True), то выполняется блок кода следующий за if.
Если условие ложно (False), то выполняется блок кода следующий за else.

Графически это можно выразить следующим образом:



© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Условный оператор elif (1)

Часто необходимо проверить несколько условий, в зависимости от истинности которых будут выполняться разные инструкции.

Для этого предусмотрен оператор **elif** (сокращение от конструкции else if), который позволяет добавить дополнительные условия в логику выполнения программы.

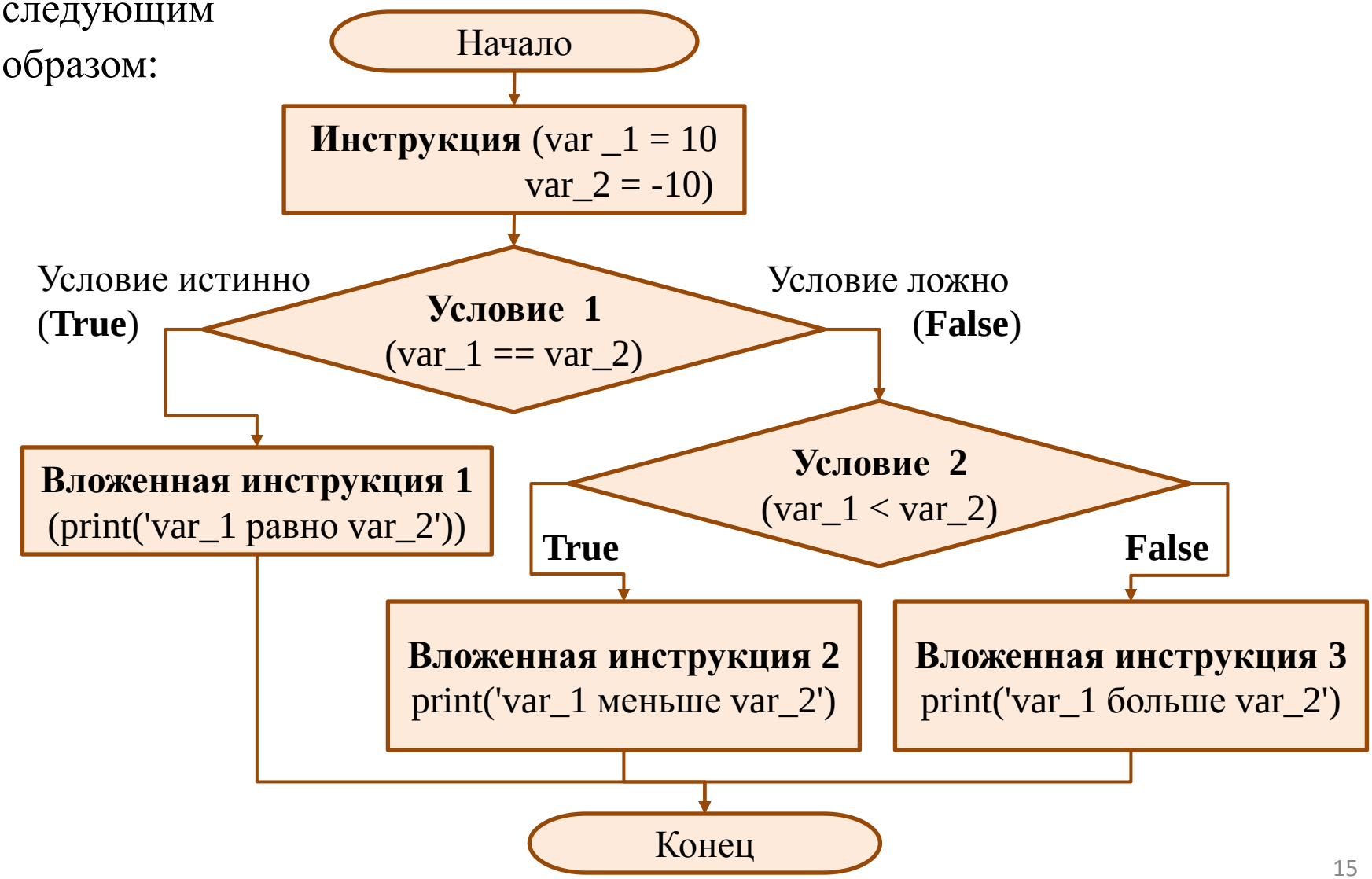
Условный оператор elif является дополнением оператора if и не используется отдельно. Размещается он между if и else, может повторяться несколько раз:

```
var_1, var_2 = 10, -10 # объявляем переменные
if var_1 == var_2: # выполняем проверку условия
    print('var_1 равно var_2') # если True
# иначе если выполняется другое условие
elif var_1 < var_2:
    print('var_1 меньше var_2') # если True для elif
else:
    print('var_1 больше var_2') # если False для всех
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Условный оператор elif (2)

Графически работу полного оператора **if-elif-else** можно выразить следующим образом:



© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Количество блоков

При написании кода необходимо учитывать, что инструкции, расположенные с **неуменьшающимся отступом** от левого края, находятся в одном блоке.

Рассмотрим пример:

```
value = input('Введите pH: ')
if value:
    pH = float(value)
    if pH == 7.0:
        print(pH, 'Вода')
    elif 7.36 < pH < 7.44:
        print(pH, 'Кровь')
    else:
        print('Какая-то жидкость')
else:
    print('Введите значение pH!')
```

В примере блоки выделены фигурными скобками.
Всего их пять, при этом блоки 3-5 являются частью блока 1.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Сокращенная форма оператора if-else

Небольшие по размеру операторы if-else (включающие только по одной вложенной инструкции) можно записывать в одну строку. В этом случае запись вида:

```
if условие:
    инструкция_1
else:
    инструкция_2
```

Преобразуется в такую:

```
инструкция_1 if условие else инструкция_2
```

Например, определим знак переменной. Для этого нужно вернуть 1, если она неотрицательна и -1 в противном случае:

```
var = 5
sign = 1 if var > 0 else -1
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева